Graphic Era deemed to be University
DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

AMBULANCE AND EMERGENCY DISPATCH SYSTEM

Team Name: *Team #*	*The Apex Achievers*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Hajariwala, neel – 2510011575* *neelhajariwala60@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*namdev, keshav – 2510360097* *namdevkeshav2212@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Chauhan,Lichhavi-2510370231* *lichhavichauhan@gmail.com* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Ahlawat,Aditya-2512510001* *adi.14ah@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

(Brief restatement of your project's main goal. Max 300 words).

The main goal of our project is to solve the issue of delayed response times during medical emergencies. In standard dispatch systems, manual call handling often leads to unnecessary delays, location errors, and miscommunication while searching for available ambulances. Our project addresses this by developing a system that connects patients, hospitals, and dispatchers directly. Using Dijkstra's shortest path algorithm and Max-Heap priority queues in C++, the backend processes incoming emergency alerts and assigns the closest available unit based on severity and distance. We are also building a simple web dashboard and mobile interface to help track vehicle status and view patient logs. By automating the assignment process and reducing manual intervention, the system helps cut down dispatch lag and speeds up emergency response.

Updated Project Approach and Architecture (2 pts)

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

Our current approach is based on a standard 3-tier architecture separating the user interface, processing backend, and storage layer.
The core dispatching logic is implemented in C++ using graph algorithms to handle routing and queue management. The web frontend is built using basic React.js components, while the mobile view is designed in Flutter for multi-platform support. Data such as driver details, hospital beds, and emergency logs are handled in a backend database.
Leaflet and Google Maps API are used to calculate real routes and show location markers.
Communication between the frontend interfaces and backend modules is handled using basic REST endpoints and WebSockets for real-time status updates.

Tasks Completed (7 pts)

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Task Completed	Team Member
Core C++ logic, project layout, graph setup, system architecture, and module integration. Implemented Max-Heap priority queue and Dijkstra's algorithm for route calculation. Created flowchart diagrams, documentation, ambulance status logging system, and basic unit testing.	Neel Hajariwala (Lead) Keshav Namdev (Developer) Lichhavi Chauhan (Developer/Docs) Aditya Ahlawat (DB/Testing)

Challenges/Roadblocks (7 pts)

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

One major issue we ran into was handling live location updates when network coverage drops mid-transit. To handle this, we are adding temporary local storage on the client side so data syncs automatically once the network reconnects. Another issue was map lag when rendering multiple moving vehicles simultaneously; we solved this by optimizing map redraw intervals and filtering idle updates. Also, standard straight-line distance calculations failed in heavy traffic areas, so we integrated road network data with dynamic routing logic. Lastly, managing continuous location tracking without excessive battery drain required tweaking device polling intervals during parked or idle states.

Tasks Pending (7 pts)

(Describe the main tasks that you still need to complete. Max 250 words).

Task Pending	Team Member (to complete the task)
Finalize web app UI layout and display live map markers. Set up WebSockets for real-time dashboard updates. Fine-tune routing code using traffic factors. Perform end-to-end integration testing and bug fixes.	Lichhavi Chauhan Keshav Namdev Neel Hajariwala Aditya Ahlawat

Project Outcome/Deliverables (2 pts)

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

The main output of this project is a functional dispatch system that automates the ambulance assignment process. Key deliverables include a working C++ backend using Dijkstra's algorithm and priority queues for fast routing, a web dashboard for dispatchers, and a mobile interface for ambulance drivers and patients. The system provides real-time tracking, structured patient logs, and hospital bed availability details in a single database, reducing manual call delays and coordination errors.

Progress Overview (2 pts)

(Summarize how much of the project is done, what's behind schedule, what's ahead of schedule. Max 200 words.)

Overall, the project is around 65% complete. The backend C++ algorithms, database schema, and core routing logic are fully implemented and ahead of schedule. However, frontend UI development and live map marker integration are slightly behind schedule due to minor API setup issues. Over the remaining weeks, our main focus will be integrating the frontend with the C++ backend and testing the complete system under different edge cases.

Codebase Information (2 pts)

(Repository link, branch, and information about important commits.)

https://github.com/LICHHAVI26/DSCPP-PBL
All members individual contribution is available in this link

Testing and Validation Status (2 pts)

(Provide information about any tests conducted)

Test Type	Status (Pass/Fail)	Notes
Priority Queue	PASS	Critical emergencies get highest priority.
Route Calculation	PASS	Dijkstra's algorithm calculates shortest routes.
Ambulance Selection	PASS	Identifies four nearest available ambulances.
Ambulance Assignment	PASS	Prevents duplicate ambulance assignment.
Hospital Selection	PASS	Displays hospital distance and available beds.
Web Dashboard	PASS	Successfully displays sample emergency results.
Input Validation	PENDING	Further testing of invalid inputs is required.

Deliverables Progress (2 pts)

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

We have completed the C++ backend modules, including priority queue management, route calculation, and database schemas for patient logs and hospital records. Currently, the web dashboard and mobile user interface are in progress. The final system integration, real-time map testing, and edge-case testing remain pending and will be completed before the final submission.